W3Schools app NEW iOS & Android
Sign In
Tutorials Exercises
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

React `useRef` Hook

‹ Previous
Next ›

The `useRef` Hook allows you to persist values between renders.

It can be used to store a mutable value that does not cause a re-render when updated.

It can be used to access a DOM element directly.

Does Not Cause Re-renders

SHARE

If we tried to count how many times our application renders using the `useState` Hook, we would be caught in an infinite loop since this Hook itself causes a re-render.

To avoid this, we can use the `useRef` Hook.

Example:

Get your own React.js Server

Use `useRef` to track application renders.

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
function App() {
  const [inputValue, setInputValue] = useState("");
  const count = useRef(0);

  useEffect(() => {
    count.current = count.current + 1;
  });

  return (
    <>
      <p>Type in the input field:</p>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h1>Render Count: {count.current}</h1>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <App />
);
```

Run Example »

`useRef()` only returns one item. It returns an Object called `current`.

When we initialize `useRef` we set the initial value: `useRef(0)`.

It's like doing this: `const count = {current: 0}`. We can access the count by using `count.current`.

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

REMOVE ADS

Accessing DOM Elements

The `useRef` Hook is often used to access DOM elements directly.

First, we create a ref using the `useRef` Hook: `const inputElement = useRef();`.

Then, we attach the ref to a DOM element using the ref attribute in JSX: `<input type="text" ref={inputElement} />`.

Finally, we can access the DOM element using the `current` property: `inputElement.current`.

Example:

Use `useRef` to focus the input:

```
import { useRef } from 'react';
import { createRoot } from 'react-dom/client';

function App() {
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
  };

  return (
    <>
      <input type="text" ref={inputElement} />
      <button onClick={focusInput}>Focus Input</button>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <App />
);
```

Run Example »

In the example, the input field gets focus when the button is clicked, because the onClick function calls `inputElement.current.focus()`.

Tracking State Changes

The `useRef` Hook can also be used to keep track of previous state values.

This is because we are able to persist `useRef` values between renders.

Example:

Use `useRef` to keep track of previous state values:

```
import { useRef, useState, useEffect } from 'react';
import { createRoot } from 'react-dom/client';
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
  useEffect(() => {
    previousInputValue.current = inputValue;
  }, [inputValue]);

  return (
    <>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h2>Current Value: {inputValue}</h2>
      <h2>Previous Value: {previousInputValue.current}</h2>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <App />
);
```

Run Example »

This time we use a combination of `useState`, `useEffect`, and `useRef` to keep track of the previous state.

In the `useEffect`, we are updating the `useRef` current value each time the `inputValue` is updated by entering text into the input field.


schools
Your next career move.
Get every certification. One payment.
CERTIFICATE OF COMPLETION
Your Name
Certified Developer
$499
Pay once.
Learn for life.
Invest in your future

REMOVE ADS

schools Tutorials Exercises W3Schools app iOS & Android NEW Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

FREE ACCOUNT

Track your progress

XP earned 0

Day streak 0

Spaces 0

League —

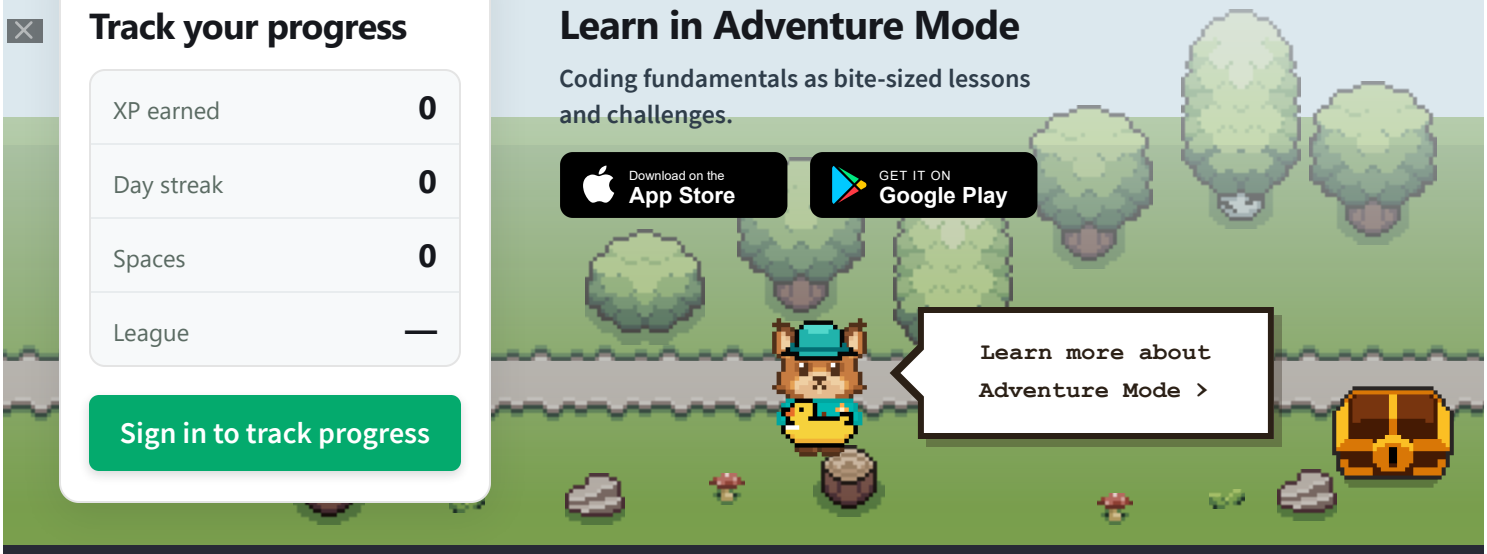
Sign in to track progress

NEW

Learn in Adventure Mode

Coding fundamentals as bite-sized lessons and challenges.

Download on the App Store GET IT ON Google Play



schools

PLUS SPACES

GET CERTIFIED FOR TEACHERS

PRACTICE CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial

W3Schools app NEW iOS & Android
Sign In
Tutorials Exercises
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
AngularJS Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

FORUM ABOUT CAMPUS

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.
Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.

Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

